

CubeCanvas: Rubik Cube Mosaic Generator

Introduction to Image Processing

Contents

1	Introduction	4
1.1	Project Objectives	4
1.2	Applications	4
2	Fundamentals of Digital Images	5
2.1	Digital Images	5
2.2	Pixel Grid	5
2.3	Grayscale Images	5
2.4	RGB Color Space	6
2.5	Why RGB Works	6
2.6	NumPy Image Representation	7
2.7	Pixel Access	7
2.8	Image Loading	7
3	Image Manipulation and Resizing	8
3.1	Image Cropping	8
3.2	Region of Interest (ROI)	8
3.3	Aspect Ratio	8
3.4	Image Resizing	9
3.5	Upsampling	9
3.6	Downsampling	9
3.7	Interpolation Techniques	9
3.7.1	Nearest Neighbor	9
3.7.2	Bilinear Interpolation	10
3.7.3	Bicubic Interpolation	10
3.7.4	Lanczos Interpolation	10
4	Color Quantization and Dithering	10
4.1	Introduction	10
4.2	Color Quantization	11
4.3	Rubik Cube Color Palette	11
4.4	RGB Color Space Geometry	12
4.5	Euclidean Distance	12
4.6	Squared Distance Optimization	13
4.7	Nearest Color Selection	13
4.8	Worked Example	14

4.9	Problem of Simple Quantization	14
4.10	Dithering	14
4.11	Floyd-Steinberg Dithering	15
4.12	Quantization Error	15
4.13	Error Diffusion Matrix	15
4.14	Error Propagation Equations	16
4.15	Python Implementation	16
4.16	Why Dithering Works	16
4.17	NumPy Processing Pipeline	17
4.18	Pixel Clipping	17
4.19	Mosaic Resolution	17
4.20	Mosaic Generation Pipeline	18
4.21	Complexity Analysis	18
5	Image Enhancement Techniques	19
5.1	Introduction	19
5.2	Image Enhancement Pipeline	20
5.3	Brightness Adjustment	20
5.4	Brightness Histogram Effect	21
5.5	Contrast Enhancement	21
5.6	Histogram Stretching	22
5.7	Color Saturation	23
5.8	Grayscale Conversion	23
5.9	Saturation Mathematics	23
5.10	Sharpness Enhancement	24
5.11	Edge Information	24
5.12	Unsharp Masking	24
5.13	Why Unsharp Masking Works	25
5.14	Convolution	25
5.15	Convolution Kernel	25
5.16	Gaussian Blur	25
5.17	Gaussian Distribution	26
5.18	Gaussian Kernel	26
5.19	Effects of Gaussian Blur	26
5.20	Frequency Domain Interpretation	26
5.21	Image Flipping	27
5.22	Horizontal Flip	27
5.23	Vertical Flip	27
5.24	Order of Operations	28
5.25	Computational Complexity	28
6	PDF Export and Mosaic Generation	29
6.1	Introduction	29
6.2	Need for Export	29
6.3	Mosaic Partitioning	29
6.4	Chunk Extraction	30
6.5	Page Generation	30
6.6	Cube Visualization	30

6.7	Overview Page	30
6.8	Coordinate Labeling	31
6.9	PDF Generation	31
6.10	Memory Management	31
6.11	Complete Workflow	31
6.12	Advantages of the System	32

1 Introduction

CubeCanvas is an image processing application designed to transform ordinary digital images into Rubik Cube mosaics.

The application combines concepts from:

- Digital Image Processing
- Computer Vision
- Numerical Computing
- Graphical User Interfaces
- Object Oriented Programming
- PDF Generation

The user imports an image, selects a crop region, applies enhancement operations, converts the image into Rubik Cube colors, and finally exports printable assembly instructions.

1.1 Project Objectives

The primary objectives are:

- Import and display images.
- Perform interactive cropping.
- Apply image enhancement techniques.
- Convert images to Rubik Cube color palettes.
- Generate mosaic representations.
- Export assembly instructions as PDF.

1.2 Applications

Applications include:

- Rubik Cube Art
- Pixel Art Creation
- Educational Demonstrations
- Color Quantization Studies
- Image Processing Projects

2 Fundamentals of Digital Images

2.1 Digital Images

A digital image is a numerical representation of a visual scene.

An image consists of tiny picture elements called pixels.

$$\textit{Pixel} = \textit{Picture Element}$$

Each pixel stores brightness and color information.

2.2 Pixel Grid

Images are arranged in rectangular grids.

For example:

$$1920 \times 1080$$

contains

$$1920 \times 1080 = 2,073,600$$

pixels.

Mathematically:

$$I(x, y)$$

represents the pixel located at coordinates

$$(x, y)$$

inside an image.

2.3 Grayscale Images

A grayscale image stores one intensity value per pixel.

$$0 \leq I(x, y) \leq 255$$

where:

- 0 = Black
- 255 = White
- Intermediate values = Shades of Gray

Example:

$$\begin{bmatrix} 0 & 50 & 100 \\ 150 & 200 & 255 \end{bmatrix}$$

2.4 RGB Color Space

Color images use three channels:

- Red
- Green
- Blue

Each channel ranges from:

$$0 \rightarrow 255$$

Therefore:

$$(R, G, B)$$

represents a pixel.

Examples:

$$(255, 0, 0)$$

Pure Red

$$(0, 255, 0)$$

Pure Green

$$(0, 0, 255)$$

Pure Blue

$$(255, 255, 255)$$

White

$$(0, 0, 0)$$

Black

2.5 Why RGB Works

Human eyes contain receptors sensitive to red, green, and blue wavelengths.

Combining these colors creates almost every visible color.

Examples:

$$(255, 255, 0)$$

Yellow

$$(255, 0, 255)$$

Magenta

$$(0, 255, 255)$$

Cyan

2.6 NumPy Image Representation

In CubeCanvas:

```
arr = np.array(image)
```

An image of size

$$300 \times 400$$

becomes

```
(300, 400, 3)
```

meaning:

- Height = 300
- Width = 400
- RGB Channels = 3

Mathematically:

$$Image \in \mathbb{R}^{H \times W \times 3}$$

2.7 Pixel Access

Individual pixels:

```
arr[y, x]
```

Example:

```
arr[10, 20]
```

returns:

```
[255, 128, 50]
```

2.8 Image Loading

Images are loaded using Pillow.

```
image = Image.open(path)
```

Supported formats:

- PNG
- JPG
- JPEG
- BMP
- TIFF

3 Image Manipulation and Resizing

3.1 Image Cropping

Cropping extracts a region from an image.

```
cropped = image.crop(  
(x1,y1,x2,y2))
```

Mathematically:

$$I_c(x, y) = I(x + x_0, y + y_0)$$

where

$$(x_0, y_0)$$

is the crop origin.

3.2 Region of Interest (ROI)

The selected crop area is called the Region of Interest.

Benefits:

- Reduces computation
- Removes unwanted regions
- Focuses processing

3.3 Aspect Ratio

Aspect Ratio:

$$AspectRatio = \frac{Width}{Height}$$

Example:

$$1920 \times 1080$$

$$\frac{1920}{1080} = 1.777$$

or

$$16 : 9$$

Maintaining aspect ratio prevents distortion.

3.4 Image Resizing

Image resizing changes dimensions.

$$I(x, y) \rightarrow I'(x', y')$$

Example:

$$1000 \times 1000$$

becomes

$$300 \times 300$$

3.5 Upsampling

Increasing image size.

Example:

$$100 \times 100 \rightarrow 1000 \times 1000$$

New pixels must be estimated.

3.6 Downsampling

Reducing image size.

Example:

$$1000 \times 1000 \rightarrow 100 \times 100$$

Information is discarded.

CubeCanvas primarily uses downsampling before mosaic generation.

3.7 Interpolation Techniques

Interpolation estimates unknown pixel values.

3.7.1 Nearest Neighbor

Used in:

`Image.Resampling.NEAREST`

Formula:

$$I'(x, y) = I(\text{round}(x), \text{round}(y))$$

Advantages:

- Fast
- Preserves sharp edges
- Good for pixel art

3.7.2 Bilinear Interpolation

Uses four neighboring pixels.

$$f(x, y) = \sum w_i p_i$$

Produces smoother results.

3.7.3 Bicubic Interpolation

Uses sixteen neighboring pixels.

Advantages:

- Better quality
- Smooth transitions
- Reduced artifacts

3.7.4 Lanczos Interpolation

Used in CubeCanvas.

`Image.Resampling.LANCZOS`

Based on the sinc function:

$$\text{sinc}(x) = \frac{\sin(\pi x)}{\pi x}$$

Lanczos Kernel:

$$L(x) = \text{sinc}(x) \cdot \text{sinc}\left(\frac{x}{a}\right)$$

Advantages:

- Excellent quality
- Sharp details
- Minimal aliasing

4 Color Quantization and Dithering

4.1 Introduction

A standard RGB image can contain millions of colors.

Since each RGB channel can take:

256

possible values, the total number of colors becomes:

$$256^3 = 16,777,216$$

colors.

However, a Rubik Cube supports only a limited set of colors.

Therefore, the image must be converted from millions of colors into a small predefined palette.

This process is called Color Quantization.

4.2 Color Quantization

Color Quantization is the process of reducing the number of colors in an image while preserving visual appearance as much as possible.

Mathematically:

$$Q(I)$$

represents the quantized version of image

$$I$$

where:

$$I \rightarrow Q(I)$$

The objective is:

$$16.7M \text{ Colors} \rightarrow 6 \text{ Rubik Colors}$$

4.3 Rubik Cube Color Palette

CubeCanvas uses six primary Rubik Cube colors.

```
palette = [  
    (255,255,255),  
    (255,255,0),  
    (255,0,0),  
    (255,136,0),  
    (0,0,255),  
    (0,170,0)  
]
```

These represent:

- White
- Yellow
- Red
- Orange
- Blue
- Green

Mathematically:

$$P = \{p_1, p_2, p_3, p_4, p_5, p_6\}$$

where:

$$p_i = (R_i, G_i, B_i)$$

4.4 RGB Color Space Geometry

RGB colors can be interpreted as points in a three-dimensional coordinate system.

Axes:

- X-axis = Red
- Y-axis = Green
- Z-axis = Blue

Each color becomes a point:

$$(R, G, B)$$

Examples:

$$(255, 0, 0)$$

Red

$$(0, 255, 0)$$

Green

$$(0, 0, 255)$$

Blue

$$(255, 255, 255)$$

White

This transforms color matching into a geometric distance problem.

4.5 Euclidean Distance

Suppose:

$$c = (R, G, B)$$

is a pixel color.

And

$$p = (r, g, b)$$

is a palette color.

Distance between them:

$$d = \sqrt{(R - r)^2 + (G - g)^2 + (B - b)^2}$$

The palette color with minimum distance is selected.

4.6 Squared Distance Optimization

Computing square roots repeatedly is expensive.

Since:

$$\sqrt{x}$$

is monotonic,

$$x_1 < x_2 \Rightarrow \sqrt{x_1} < \sqrt{x_2}$$

therefore:

$$(R - r)^2 + (G - g)^2 + (B - b)^2$$

can be compared directly.

This optimization is used in the project.

4.7 Nearest Color Selection

Implementation:

```
def nearest_color(color, palette):  
  
    distances =  
    np.sum(  
        (palette - color)**2,  
        axis=1)  
  
    return palette[  
        np.argmin(distances)]
```

Algorithm:

1. Compute distance to every palette color.
2. Store distances.
3. Find minimum distance.
4. Return nearest palette color.

4.8 Worked Example

Consider:

$$c = (250, 20, 20)$$

Distance from red:

$$(255, 0, 0)$$

$$\begin{aligned}d_r^2 &= (250 - 255)^2 + (20 - 0)^2 + (20 - 0)^2 \\&= 25 + 400 + 400 = 825\end{aligned}$$

Distance from white:

$$\begin{aligned}d_w^2 &= (250 - 255)^2 + (20 - 255)^2 + (20 - 255)^2 \\&= 25 + 55225 + 55225 = 110475\end{aligned}$$

Since:

$$825 < 110475$$

the pixel becomes red.

4.9 Problem of Simple Quantization

Suppose a grayscale gradient:

$$0 \rightarrow 255$$

After quantization:

$$0 \rightarrow 0 \rightarrow 255 \rightarrow 255$$

Visible color bands appear.

This effect is called:

Color Banding

Banding reduces image quality.

4.10 Dithering

Dithering is used to reduce banding artifacts.

Instead of independently replacing each pixel, quantization errors are distributed to neighboring pixels.

Advantages:

- Smoother gradients
- Better visual quality
- Reduced banding